

Functional Specifications Document

July 8, 2024



Ambient Systems

A business software to help companies improve their financial performance by enabling finance teams to connect data from various systems and plan better and accurately.

Table of Contents

- Product Pages
- Features
- Development Tasks
-
- Data Model
- Service Delivery Process (TBD)

(TBD)



Product Pages
Case Scenario: Vertical Farm or Commercial Real Estate Property

Pages Designed



Overview

Financial Metrics

Data Status

Operational Metrics

Recommendations

Camera Feed



Request

AI Request

Report Request

Hardware Request

API Connection Request

Request



Data

Data Library



Finance

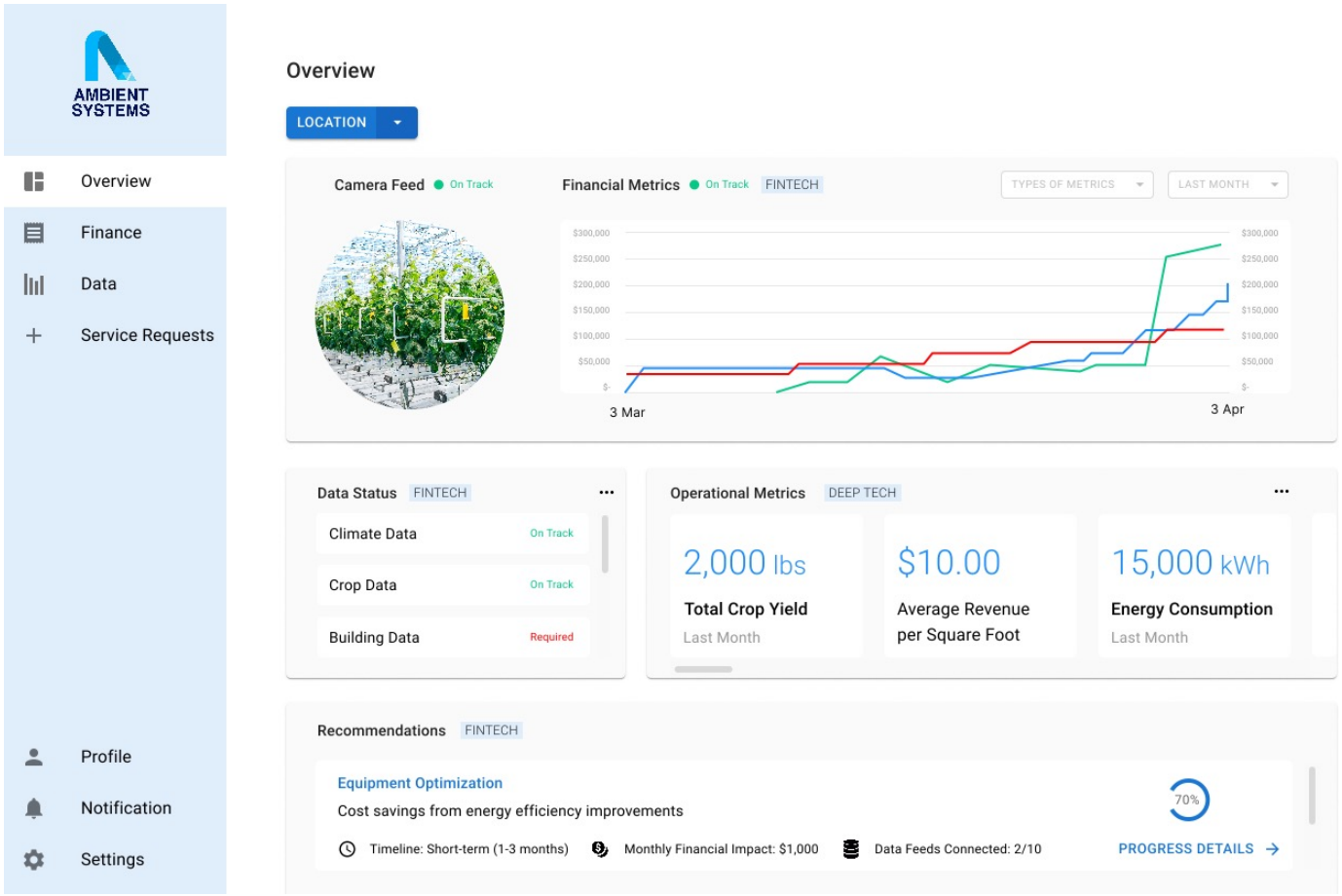
Planning

Data Selection

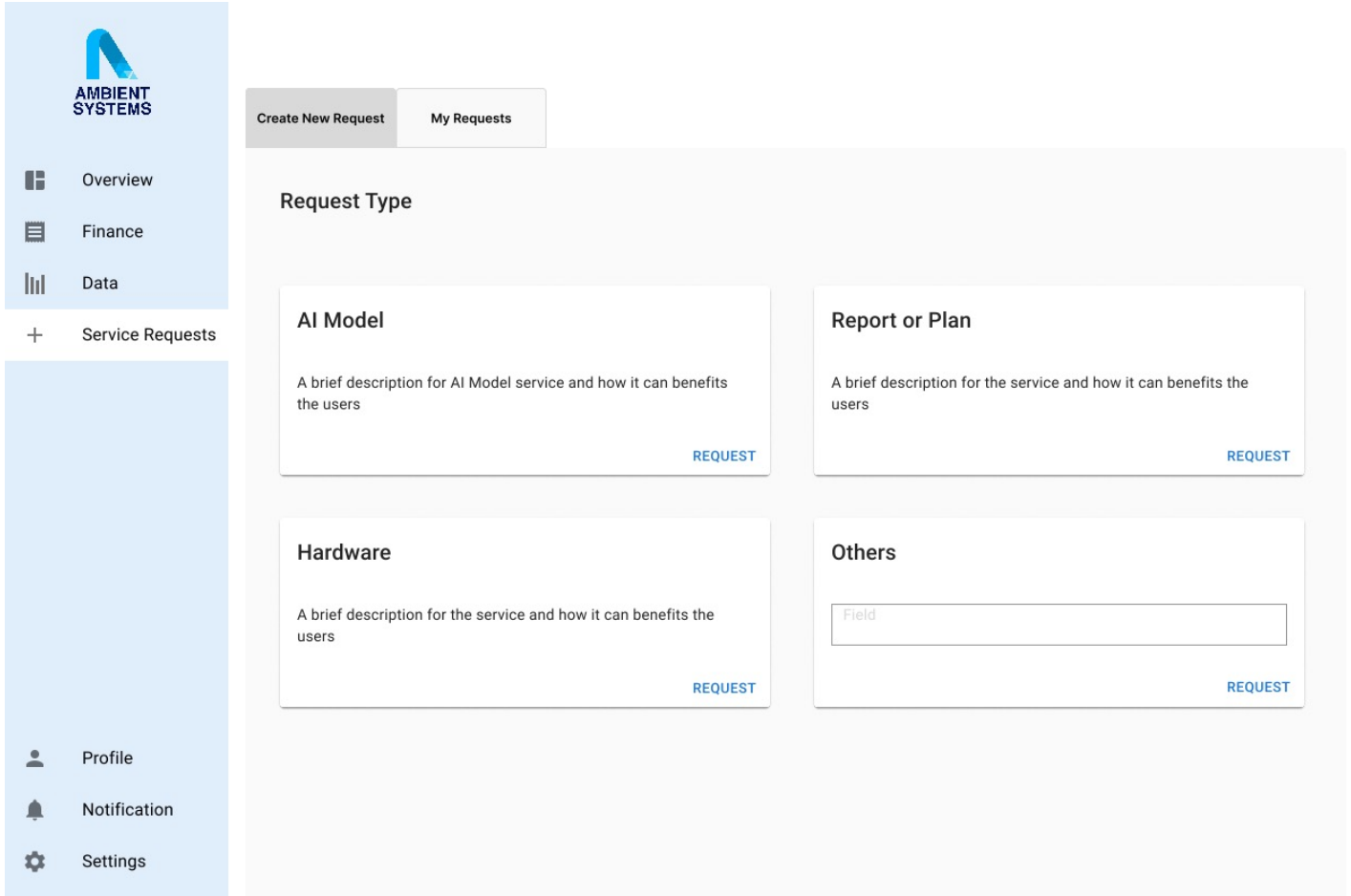
Goals List

Planning

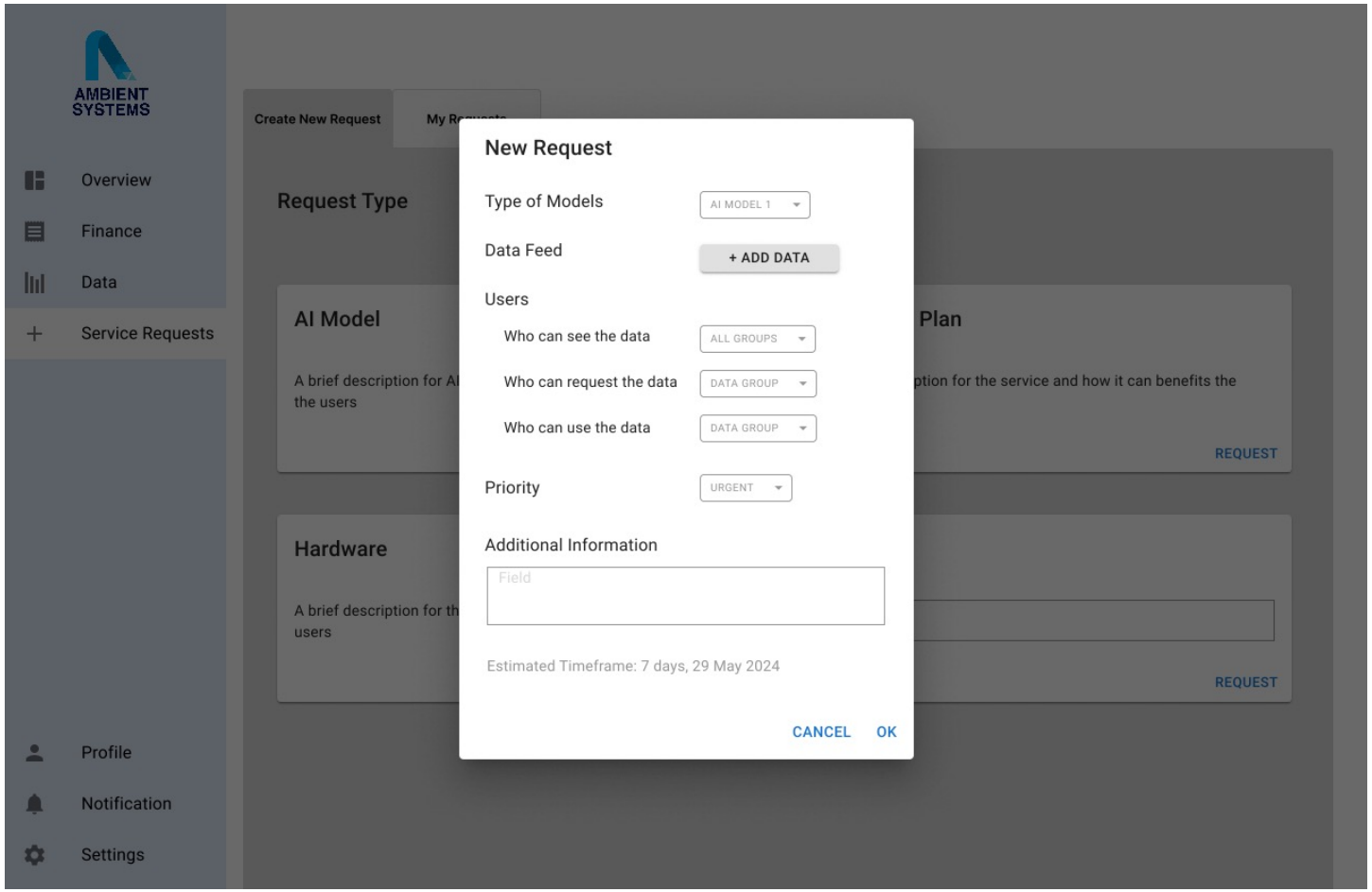
Overview Page



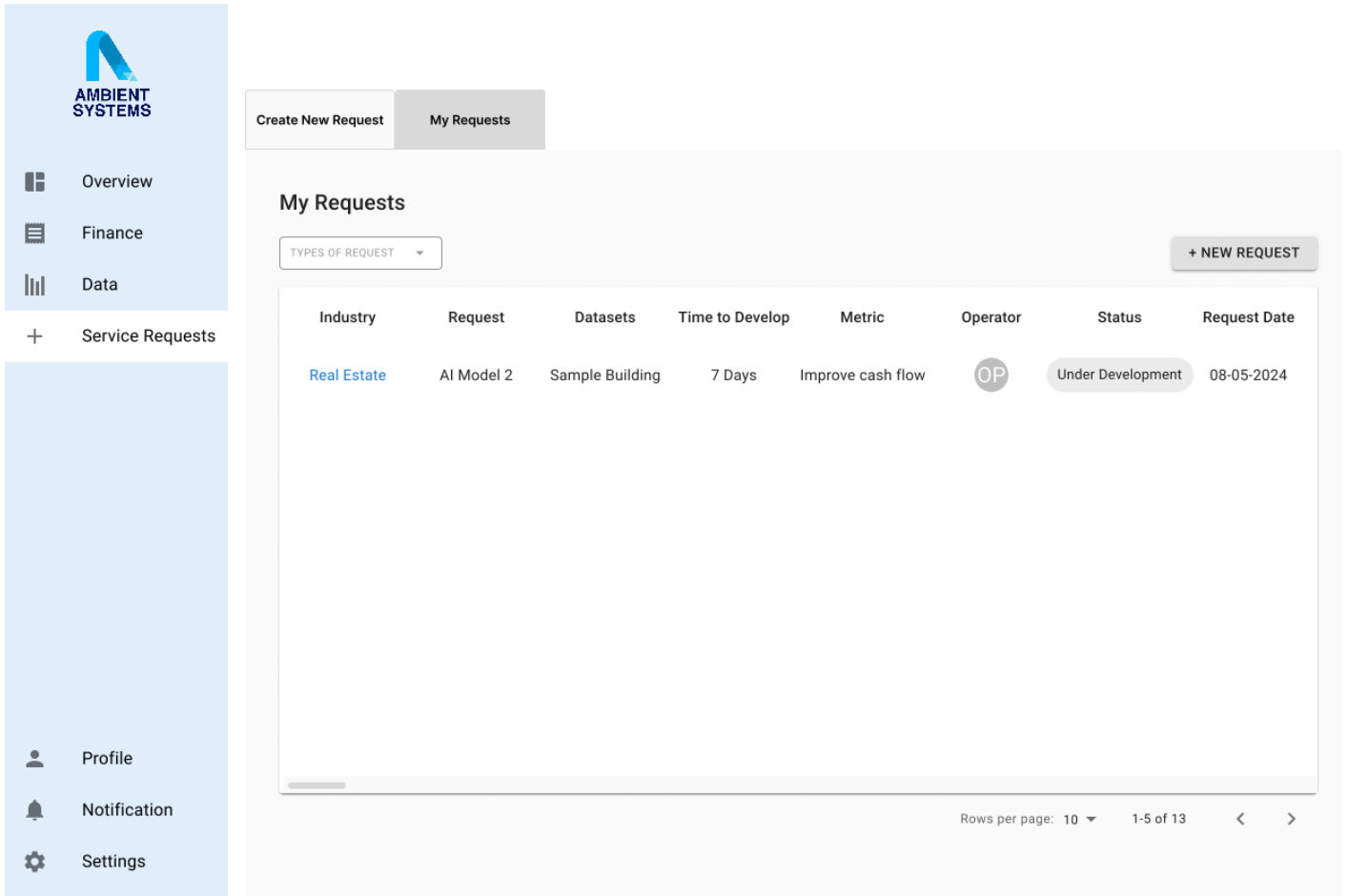
Request Page: New Request



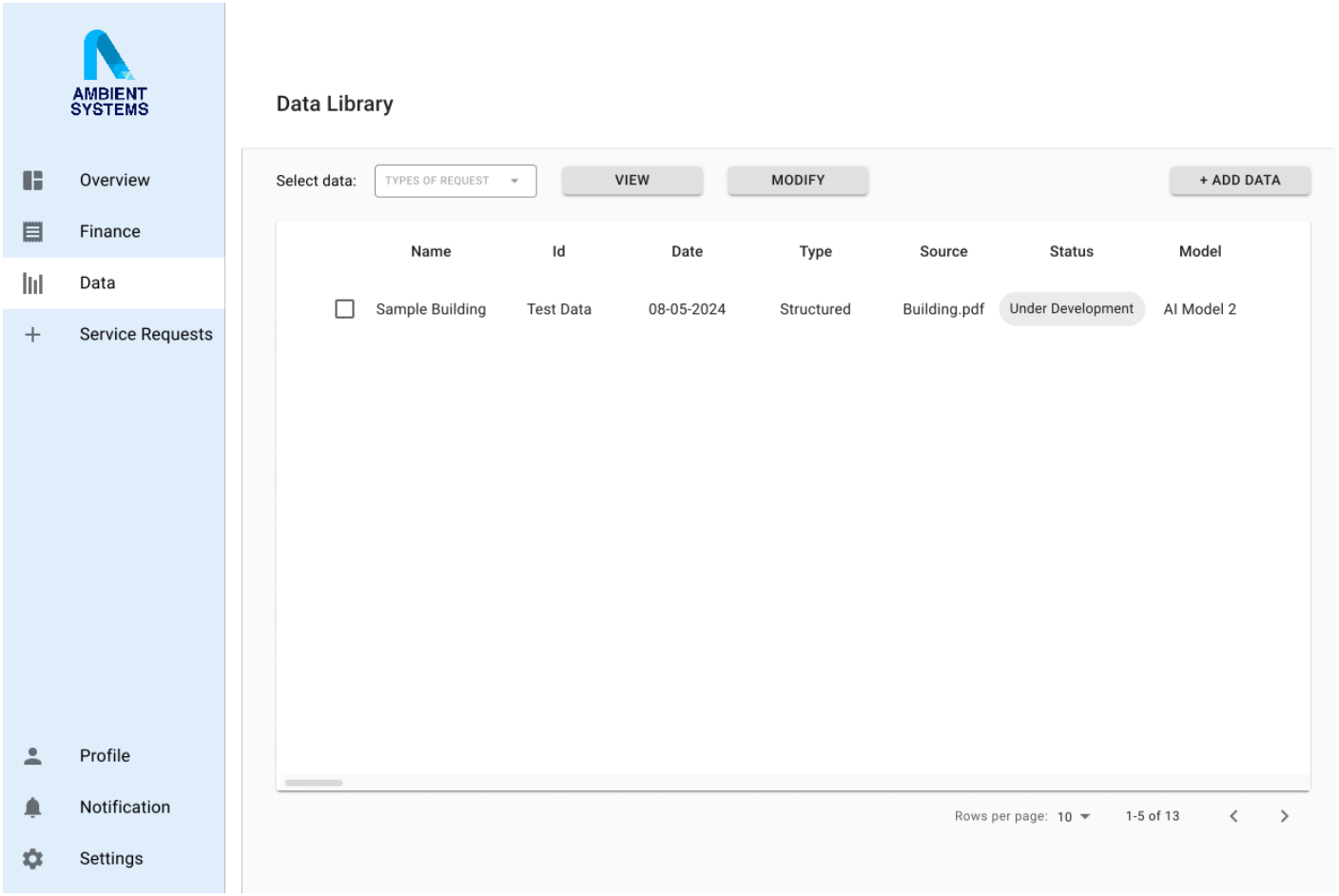
Request Page: New Request



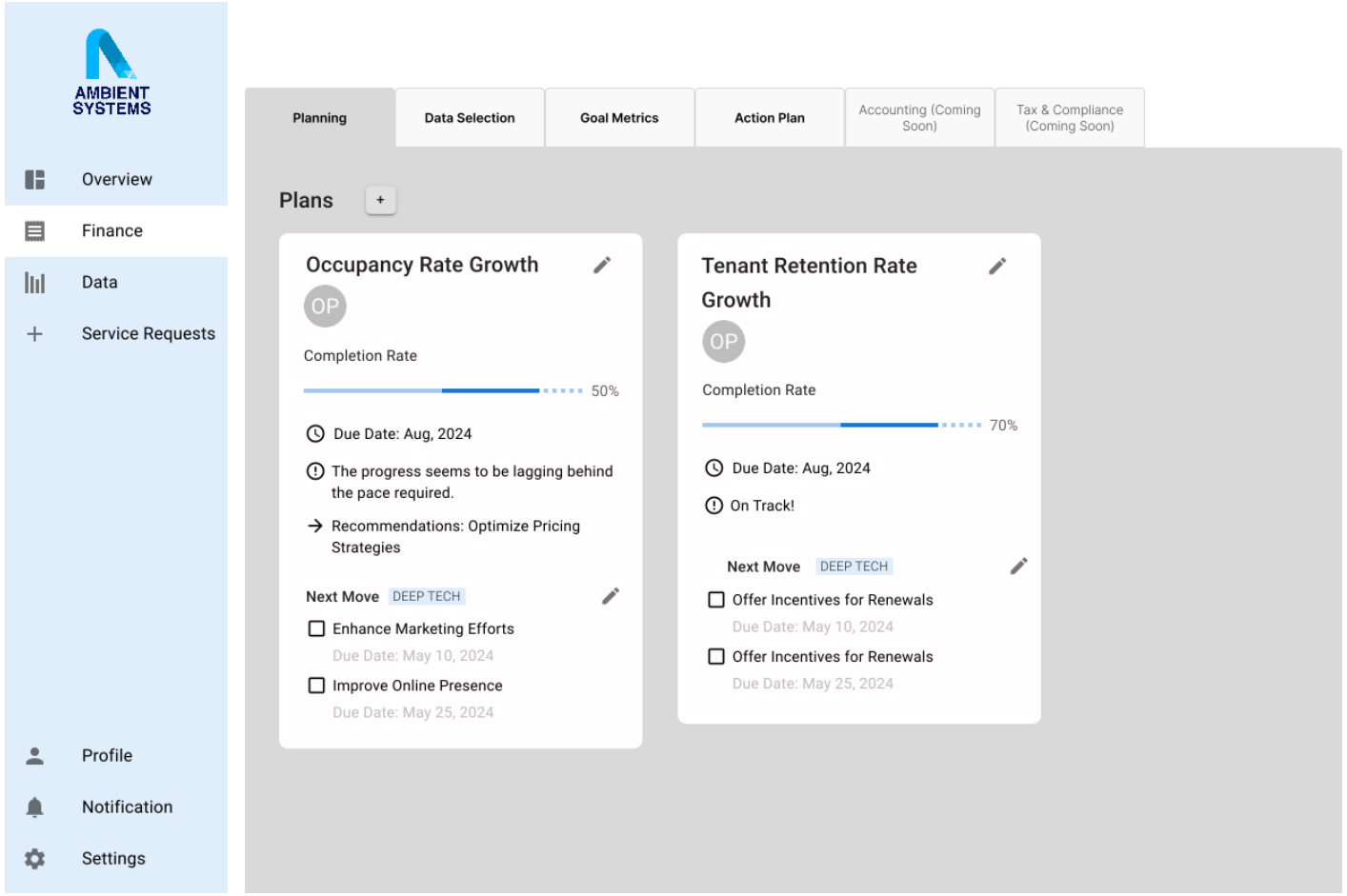
Request Page: My Requests



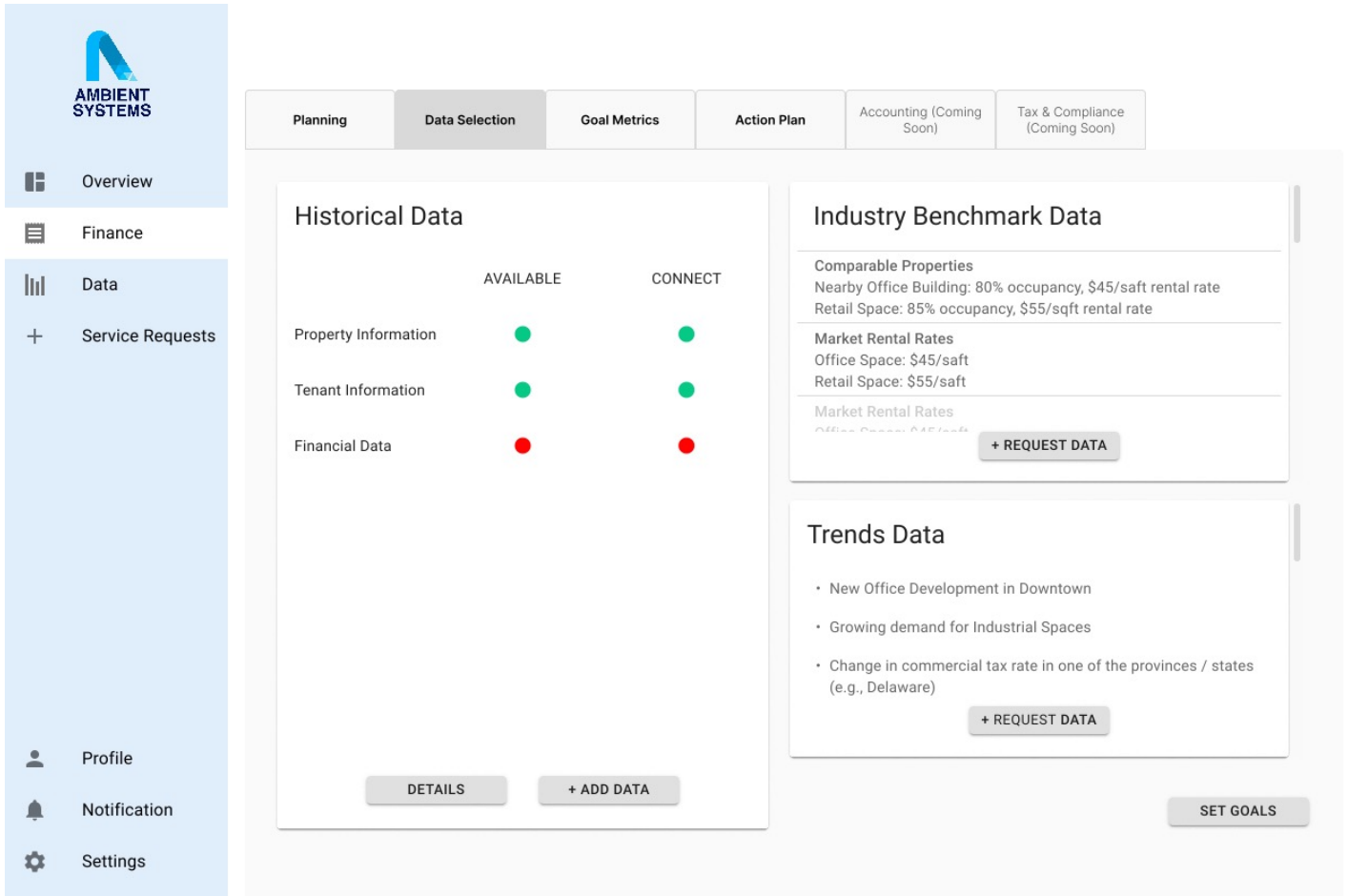
Data Page: Data Library



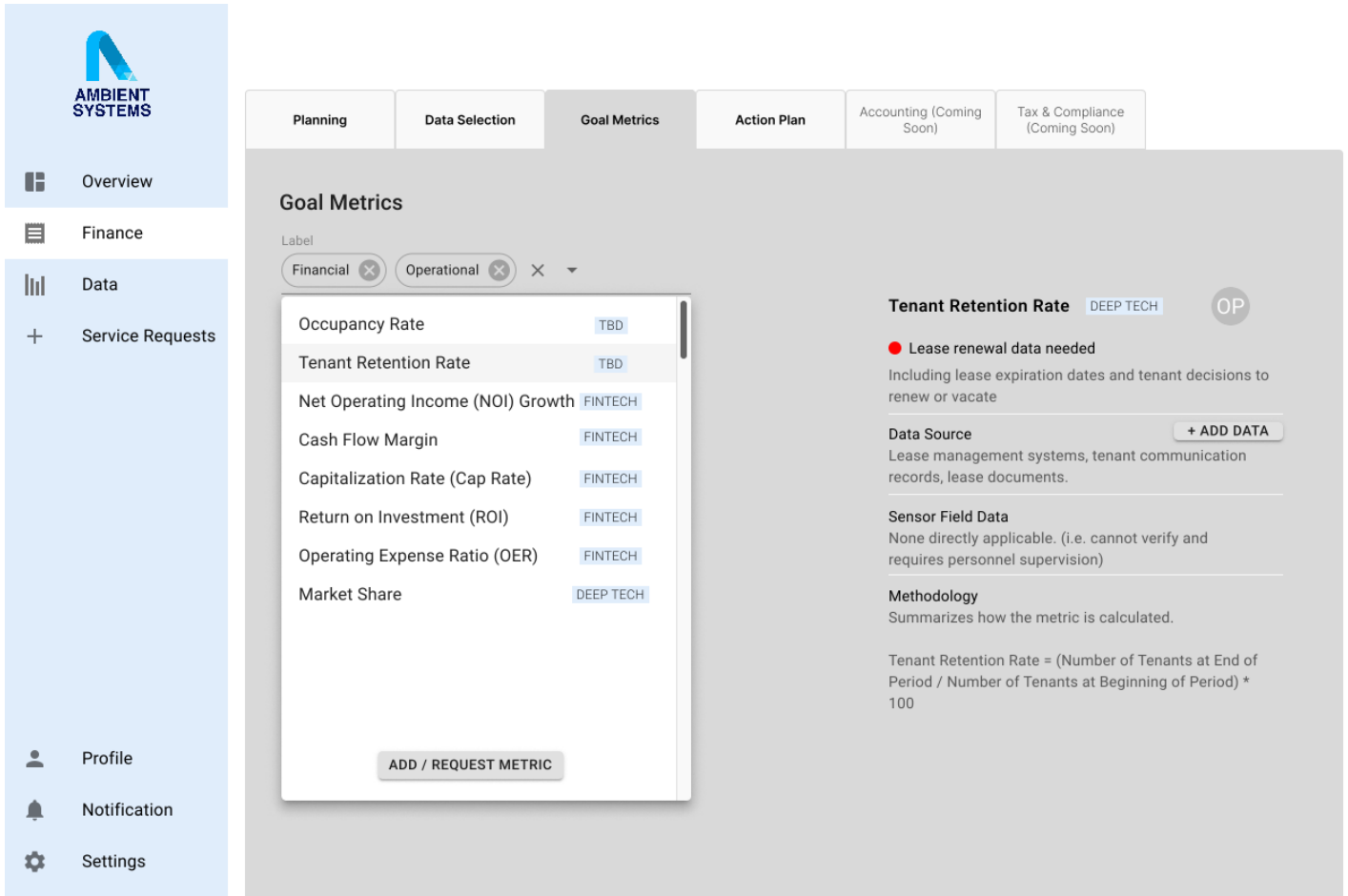
Finance Page: Planning



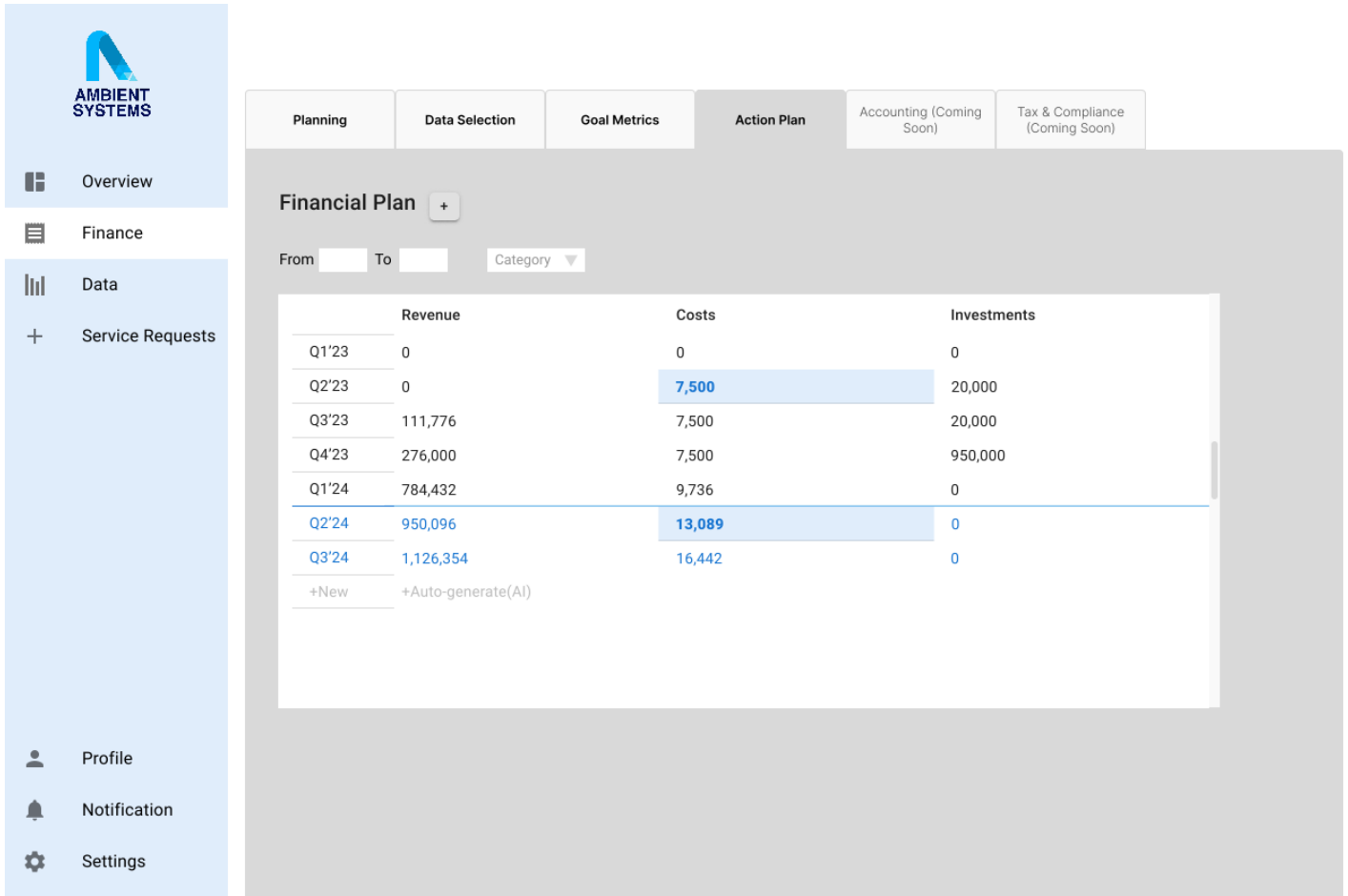
Finance Page: Data Selection



Finance Page: Goal Metrics



Finance Page: Action Plan





Product Features

Product Features

	Features	Ambient Core	Ambient Ascend	Ambient Ascend IA
1	Cloud-Based Web Platform	✓	✓	✓
2	Financial Metrics	✓	✓	✓
3		✓	✓	✓
4	Dashboard Visualization	✓	✓	✓
5	Export (pdf, csv)	✓	✓	✓
6	Financial Plan Inputs	✓	✓	✓
7	Operational Metrics	X	✓	✓
8	Data connection	X	✓	✓
9	AI-supported Data Verification	X	X	✓
10	AI-supported Financial Planning	X	X	✓
	Pricing per Month per User	**$50.00**	**$200.00**	**$400.00**
	Pricing per Month (5 users) 25% discount	**$187.50**	**$750.00**	**$1500.00**


Development Tasks

Core Features

Cloud-Based Web Platform

1. **Setup Hosting Environment**
    - Configure cloud servers
    - Setup databases
    - Implement security protocols
2. **Develop Core Architecture**
    - Design system architecture
    - Implement microservices
    - Setup API gateways
3. **User Authentication and Authorization**
    - Implement login/signup functionality
    - Setup user roles and permissions
4. **Frontend Development**
    - Create UI/UX design
    - Develop responsive web pages
5. **Backend Development**
    - Develop RESTful APIs
    - Implement data models and business logic

Financial Metrics

1. **Requirement Analysis**
   - Identify key financial metrics
   - Gather user requirements
2. **Data Collection and Integration**
   - Integrate with financial data sources
   - Develop data aggregation logic
3. **Frontend Development**
   - Design metric visualizations (charts, graphs)
   - Implement metric display on dashboards
4. **Backend Development**
   - Develop data processing algorithms
   - Implement API endpoints for metric retrieval

1. **Requirement Analysis**
   - Identify data formats and sources
2. **Frontend Development**
   - Design UI
   - Implement file upload functionality
3. **Backend Development**
   - Develop data parsing logic
   - Implement data validation and error handling
4. **Data Storage**
   - Setup database schemas for uploaded data
   - Implement data storage logic

Dashboard Visualization

1. **Requirement Analysis**
   - Identify key metrics and data to visualize
2. **Frontend Development**
   - Design
   - Implement visualization components (charts, tables)
3. **Backend Development**
   - Develop API endpoints for data retrieval
   - Implement data aggregation and processing logic

Export (pdf, csv)

1. **Requirement Analysis**
   - Identify export formats and data points
2. **Frontend Development**
   - Design export options in the UI
3. **Backend Development**
   - Implement data formatting logic
   - Develop export functionality (pdf, csv)

Financial Plan Inputs

1. **Requirement Analysis**
   - Identify required financial inputs and calculations
2. **Frontend Development**
   - Design input forms and UI components
3. **Backend Development**
   - Implement data processing and validation logic
   - Develop storage and retrieval mechanisms for financial plans

Enhanced Analytics

Operational Metrics

1. **Requirement Analysis**
   - Identify key operational metrics
2. **Data Collection and Integration**
   - Integrate with operational data sources
   - Develop data aggregation logic
3. **Frontend Development**
   - Design operational metric visualizations
   - Implement metric display on dashboards
4. **Backend Development**
   - Develop data processing algorithms
   - Implement API endpoints for metric retrieval

Data Connection

1. **Requirement Analysis**
   - Identify external data sources
2. **Integration Development**
   - Implement data connectors and adapters
   - Setup data synchronization mechanisms
3. **Backend Development**
   - Implement data ingestion and processing logic
   - Develop API endpoints for data access

AI and Advanced Analytics

AI-supported Data Verification

1. **Requirement Analysis**
   - Identify verification criteria and AI algorithms
2. **Model Development**
   - Develop and train AI models
   - Implement data preprocessing and feature extraction
3. **Backend Development**
   - Integrate AI models with backend systems
   - Implement API endpoints for data verification
4. **Frontend Development**
   - Design UI for data verification results
   - Implement visualization of verification outcomes

AI-supported Financial Planning

1. **Requirement Analysis**
   - Identify planning algorithms and AI models
2. **Model Development**
   - Develop and train AI models for financial planning
   - Implement scenario analysis and forecasting
3. **Backend Development**
   - Integrate AI models with backend systems
   - Develop API endpoints for financial planning
4. **Frontend Development**
   - Design UI for financial planning tools
   - Implement visualization of planning outcomes

User Management

Settings and Profile Permissions

1. **Requirement Analysis**
   - Identify settings and permission requirements
2. **Frontend Development**
   - Design settings and profile UI
   - Implement forms and controls for settings
3. **Backend Development**
   - Develop logic for managing settings and permissions
   - Implement API endpoints for settings management



Jira Software

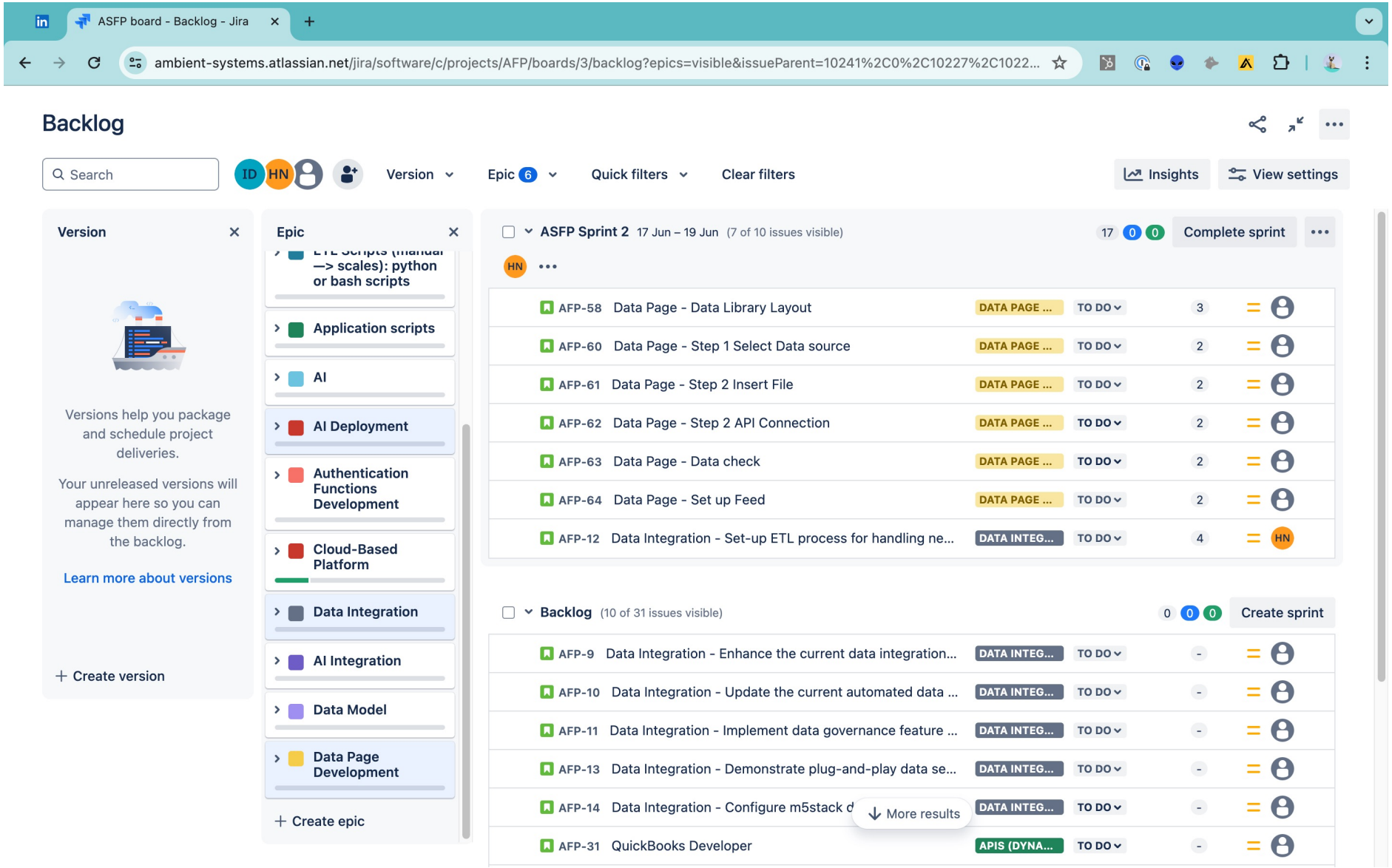
Summary

- Keep track of different types of **issues**, such as bugs or tasks.
- Each issue type can be configured differently.

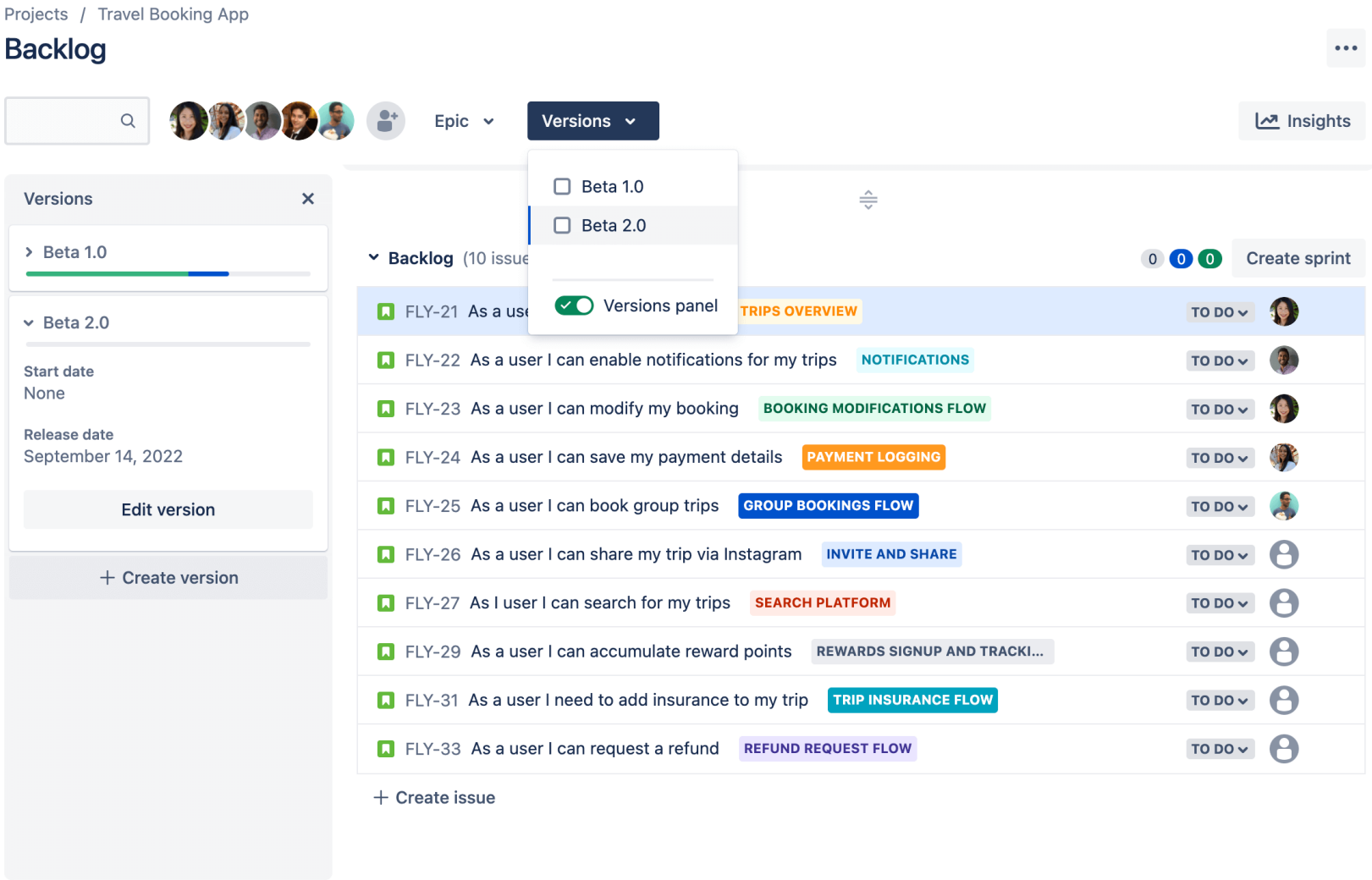
Icon	Issue Type	Description
	Story	Functionality or a feature expressed as a user goal. A story can usually have only one epic.
	Bug	A problem or error.
	Epic	A big user story that needs to be broken down. Created by - do not edit or delete.
	Task	A small, distinct piece of work.
	Subtask	A small piece of work that's part of a larger task.

Epics and Stories

- Jira is a software tool for managing development tasks
- It consists of using versions, epics, and stories
- **Epics:** 
  - Create an Epic for each main feature or page.
- **Stories:** 
  - Within each Epic, create Stories for individual tasks or components.
  - Each Story should include detailed descriptions, acceptance criteria, and time estimates as specified in the FSD.
- **Versions**

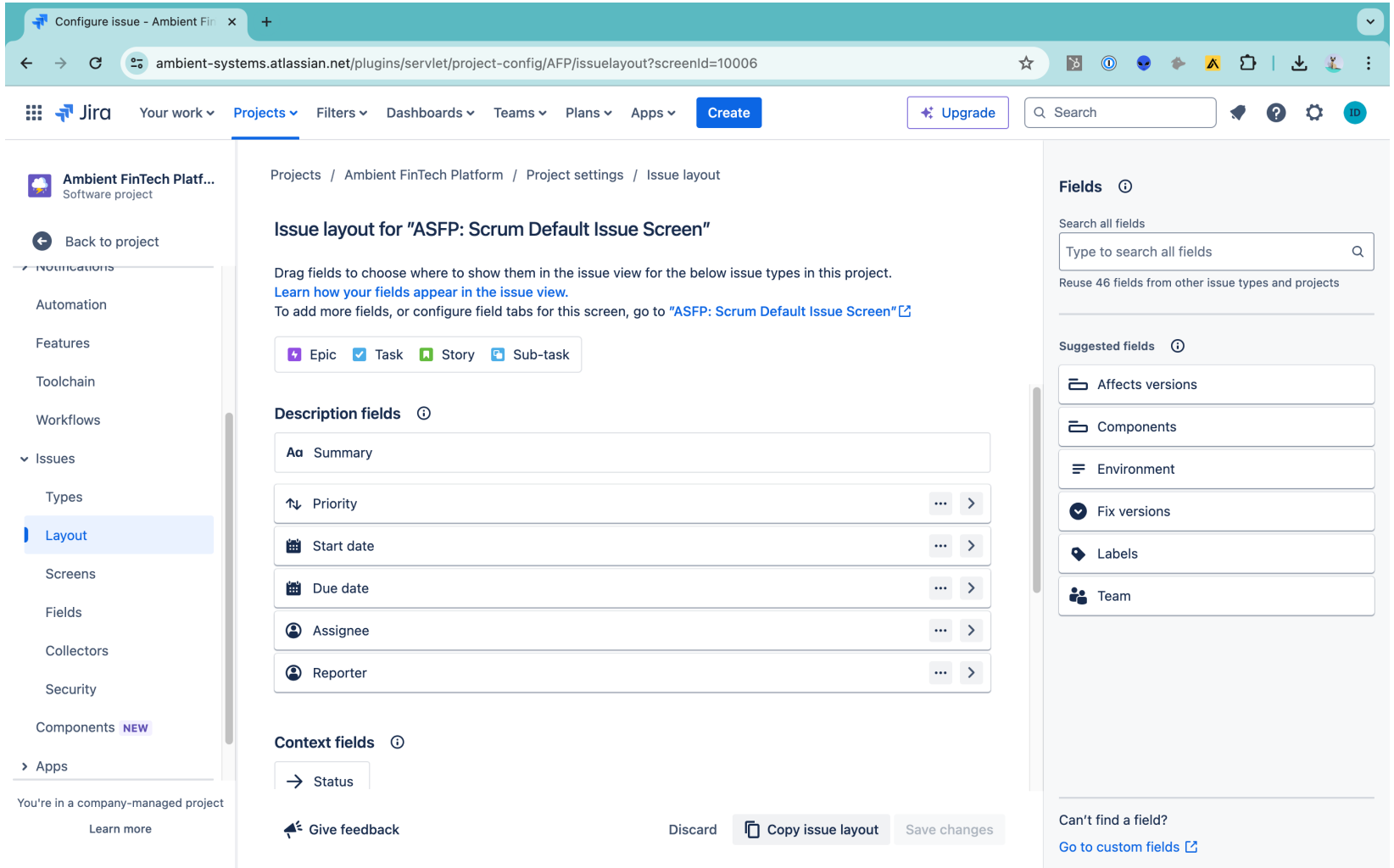

ASFP board - Backlog - Jira
ambient-systems.atlassian.net/jira/software/c/projects/AFP/boards/3/backlog?epics=visible&issueParent=10241%2C0%2C10227%2C1022...
Backlog
Search
ID HN
Version
Epic 6
Quick filters
Clear filters
Insights
View settings
Version
Versions help you package and schedule project deliveries.
Your unreleased versions will appear here so you can manage them directly from the backlog.
Learn more about versions
+ Create version
Epic
—> scales): python or bash scripts
Application scripts
AI
AI Deployment
Authentication Functions Development
Cloud-Based Platform
Data Integration
AI Integration
Data Model
Data Page Development
+ Create epic
ASFP Sprint 2 17 Jun – 19 Jun (7 of 10 issues visible)
17 0 0
Complete sprint
AFP-58 Data Page - Data Library Layout DATA PAGE ... TO DO 3
AFP-60 Data Page - Step 1 Select Data source DATA PAGE ... TO DO 2
AFP-61 Data Page - Step 2 Insert File DATA PAGE ... TO DO 2
AFP-62 Data Page - Step 2 API Connection DATA PAGE ... TO DO 2
AFP-63 Data Page - Data check DATA PAGE ... TO DO 2
AFP-64 Data Page - Set up Feed DATA PAGE ... TO DO 2
AFP-12 Data Integration - Set-up ETL process for handling ne... DATA INTEG... TO DO 4 HN
Backlog (10 of 31 issues visible)
0 0 0
Create sprint
AFP-9 Data Integration - Enhance the current data integration... DATA INTEG... TO DO -
AFP-10 Data Integration - Update the current automated data ... DATA INTEG... TO DO -
AFP-11 Data Integration - Implement data governance feature ... DATA INTEG... TO DO -
AFP-13 Data Integration - Demonstrate plug-and-play data se... DATA INTEG... TO DO -
AFP-14 Data Integration - Configure m5stack d DATA INTEG... TO DO -
More results
AFP-31 QuickBooks Developer APIS (DYNA... TO DO -

Versions


Projects / Travel Booking App
Backlog
Epic
Versions
Insights
Beta 1.0
Beta 2.0
Versions panel
Versions
Beta 1.0
Beta 2.0
Start date
None
Release date
September 14, 2022
Edit version
Create version
Backlog (10 issue
0 0 0
Create sprint
FLY-21 As a use TRIPS OVERVIEW TO DO
FLY-22 As a user I can enable notifications for my trips NOTIFICATIONS TO DO
FLY-23 As a user I can modify my booking BOOKING MODIFICATIONS FLOW TO DO
FLY-24 As a user I can save my payment details PAYMENT LOGGING TO DO
FLY-25 As a user I can book group trips GROUP BOOKINGS FLOW TO DO
FLY-26 As a user I can share my trip via Instagram INVITE AND SHARE TO DO
FLY-27 As I user I can search for my trips SEARCH PLATFORM TO DO
FLY-29 As a user I can accumulate reward points REWARDS SIGNUP AND TRACKI... TO DO
FLY-31 As a user I need to add insurance to my trip TRIP INSURANCE FLOW TO DO
FLY-33 As a user I can request a refund REFUND REQUEST FLOW TO DO
Create issue

Fields

- Fields capture and display the information people need to carry out a project's work.
- Types of Fields:
  - **Description fields**
  - **Context fields**
  -



Field Types ( )

- Checkboxes: choose multiple values using checkboxes
- Date and Date Time Picker: stores dates or date and time
- Label: arbitrary labels to issues
- Number field: stores and validates numeric (floating point) input
- Radio buttons: a list of radio buttons
- Select list (cascading, multiple choice, single choice): choose multiple values using two select lists, multiple values in a select list, or a single select list with configurable list options.

  paragraph (plain text): a single line of plain text
- URL field: input a single URL
- User picker (single user)

https://support.atlassian.com/jira-cloud-administration/docs/custom-fields-types-in-company-managed-projects/

Advanced Field Types ( )

- Checkbox
- Custom formula
- Date of First Response
- Days since last comment
- Domain of Assignee
- Domain of Reporter
- External asset platform
- Global rank
- Group picker (multi-group or single group)
- Last commended by
  - public comment date
    - custom field (for edit or for view)
- Number of attachments
- Number of comments
- Participants of an issue
- Project picker (single Project)
- Rating
- Reactions
- Slide
- Text Field (read only)
- Time in Status
- Time Interval
- User Picker
- User Property Field
- Username of last updater or commenter
- Version. Picker (multiple versions or single version)

https://support.atlassian.com/jira-cloud-administration/docs/custom-fields-types-in-company-managed-projects/

Issue AFP-58 (Story)

Data Page Development (AFP-59) [AFP-58] Data Page - Data Library Layout Created: 18/Jun/24 Updated: 18/Jun/24 Resolved: 18/Jun/24

Status:	Done
Project:	Ambient FinTech Platform
Components:	None
Affects versions:	None
Fix versions:	None
Parent:	Data Page Development

Type:	Story	Priority:	Medium
Reporter:		Assignee:	
Resolution:	Done	Votes:	0
Labels:	None		
Remaining Estimate:	Not Specified		
Time Spent:	Not Specified		
Original estimate:	Not Specified		

Sprint:	ASFP Sprint 2

AFP-59 / AFP-58

1

Data Page - Data Library Layout

Attach | Create subtask | Link issue | Record a video

Priority: Medium

Start date: None

Due date: None

Assignee: HN Hilary Ng

Assign to me

Reporter: HN Hilary Ng

Confluence pages

Product requirements TRY TEMPLATE — Updated just now

Or explore more templates

ID Add a comment...

Pro tip: press M to comment

Done | Done | Actions

Details

Original estimate: 0m

Time tracking: No time logged

Sprint: ASFP Sprint 2

Parent: AFP-59 Data Page Developm...

Description

Dialouge:

Step 1: Select Data Source

Step 2: Insert File/ API Connection

Data Check

Set up Feed

Development: Create branch

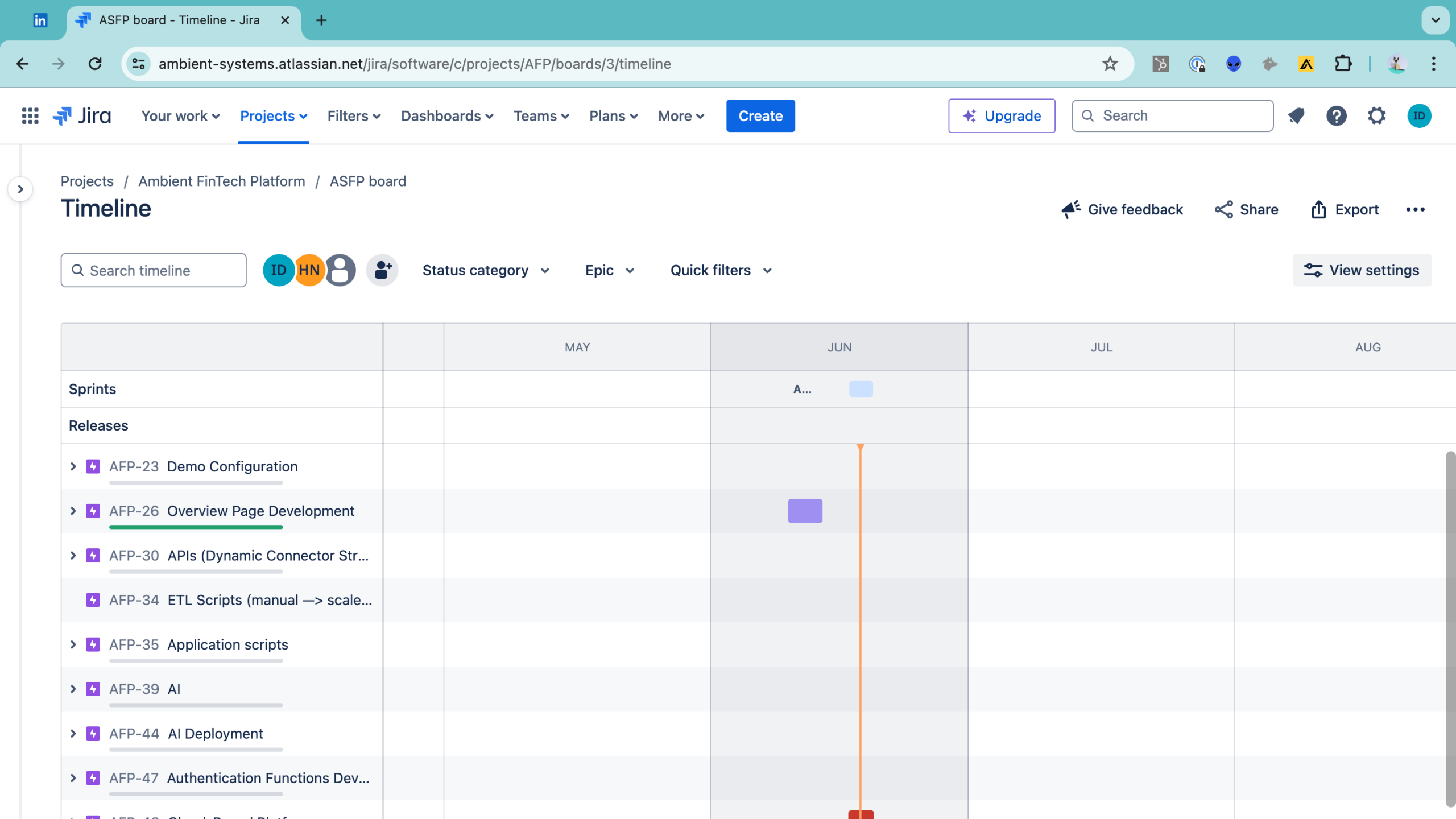
1 commit 27 minutes ago

Created June 18, 2024 at 2:29 PM

Configure

AUTHENTICATION FUNCTIONS DEVELOPMENT

AFP-2 2 HN


ASFP board - Timeline - Jira
ambient-systems.atlassian.net/jira/software/c/projects/AFP/boards/3/timeline
Jira
Your work
Projects
Filters
Dashboards
Teams
Plans
More
Create
Upgrade
Search
Projects / Ambient FinTech Platform / ASFP board
Timeline
Give feedback
Share
Export
Search timeline
Status category
Epic
Quick filters
View settings
MAY
JUN
JUL
AUG
Sprints
Releases
AFP-23 Demo Configuration
AFP-26 Overview Page Development
AFP-30 APIs (Dynamic Connector Str...
AFP-34 ETL Scripts (manual —> scale...
AFP-35 Application scripts
AFP-39 AI
AFP-44 AI Deployment
AFP-47 Authentication Functions Dev...

ASFP Sprint 2

0 days | Complete sprint

Search | ID HN | Epic | Quick filters | GROUP BY Stories | Insights | View settings

TO DO 9

Data Page - Data Library Layout
DATA PAGE DEVELOPMENT
AFP-58 3

Data Page - Step 1 Select Data source
DATA PAGE DEVELOPMENT
AFP-60 2

Data Page - Step 2 Insert File
DATA PAGE DEVELOPMENT
AFP-61 2

Data Page - Step 2 API Connection
DATA PAGE DEVELOPMENT
AFP-62 2

Data Page - Data check
DATA PAGE DEVELOPMENT
AFP-63 2

IN PROGRESS

DONE 1

Cloud-Based Platform - OVH Cloud Storage Option Research
CLOUD-BASED PLATFORM
~~AFP-57~~ 2 HN

Ambient FinTech Platf...
Software project

PLANNING

ASFP board
Board

Timeline
Backlog
Active sprints
Reports

List
Goals
Issues
Components

DEVELOPMENT

Code
Security
Releases

Code

Give feedback

Latest repositories

The most recent repositories with commits, branches, or pull requests linked to this project.

Showing 2 of 2.

Bitbucket connected

Not seeing your repositories? Use issue keys to link them or check the app configuration

GitHub

Ambient-Team/ambient-...

Last updated 6 days ago

Linked pull requests

Your team's linked pull requests from the last 30 days. Learn more

Search pull requests | Repository | Author | Reviewer | Status

Summary	Status	Reviewers	Comments	Updated
AFP-26 | Overview Page Development → main #2 • Repository: Ambient-Team/ambient-fintech	MERGED	*No reviewers*	0	6 days ago
AFP-22 Implement Initial Overall Layout and Navigation Bar → main #1 • Repository: Ambient-Team/ambient-fintech	MERGED	*No reviewers*	0	8 days ago

duct Features, , and

- **Product Features to Epics:**
  - Each product feature is mapped to one or more epics in Jira.
  - An epic represents the overall implementation of a feature.
- **to Stories:**
  - Each page design corresponds to multiple stories. These stories cover all the functionalities, UI components, and user interactions described in the design.
  - Stories are grouped under the relevant epic(s).
- **Stories to Tasks/Sub-Tasks:**
  - Each story is broken down into tasks or sub-tasks representing individual development steps, including backend, frontend, testing, and integration tasks.

Example: Feature-based Tasks

- **Product Feature: Cloud-Based Web Platform**
- **Epic:** Cloud-Based Web Platform
- **Stories:**
  - User Authentication and Authorization
    - Task: Implement login/signup functionality
    - Task: Setup user roles and permissions
  - Develop Core Architecture
    - Task: Design system architecture
    - Task: Implement microservices
    - Task: Setup API gateways
  - Frontend Development
    - Task: Create UI/UX design
    - Task: Develop responsive web pages
  - Backend Development
    - Task: Develop RESTful APIs
    - Task: Implement data models and business logic

Example: Page-based Tasks

- **Page Design: Overview Page**
- **Epic:** Overview Page
- **Stories:**
  - Display Financial Metrics
    - Task: Fetch financial data from API
    - Task: Create visualization components (charts, graphs)
  - Display Operational Metrics
    - Task: Integrate with operational data sources
    - Task: Develop data aggregation logic
  - Display Recommendations
    - Task: Implement recommendation logic
    - Task: Design UI for recommendations

Pages Linked to Feature

Feature	Pages
Web Platform	Overview, Financial Metrics, Data Status, Operational Metrics, Recommendations, Data Library, Finance (Planning, Data Selection, Goals List), Settings and Profile Permissions
Financial Metrics	Overview, Financial Metrics
	Request (AI, Report, Hardware, API Connection,), Data Library
Dashboard Visualization	Overview, Financial Metrics, Operational Metrics, Recommendations
Export (pdf, csv)	Request (Report), Data Library
Financial Plan Inputs	Finance (Planning, Goals List)
Operational Metrics	Overview, Operational Metrics
Data connection	Overview, Data Library
AI-supported Data Verification	Request (AI), Data Library
AI-supported Financial Planning	Finance (Planning)
Settings and Profile Permissions	Settings and Profile Permissions

Explanation: Each feature is associated with specific pages where it is utilized or relevant functionalities are provided. This table provides a comprehensive view of how each feature integrates with different parts of the application, including data management, user requests, financial planning, and settings management.

Pages Linked to Features Linked to Jira (1/2)

Feature Number	Feature	Pages	and Stories
1	Cloud-Based Web Platform	Overview, Financial Metrics, Data Status, etc.	Epic: Cloud-Based Web Platform Stories: User Authentication, Core Architecture, Frontend Development, Backend Development
2	Financial Metrics	Overview, Financial Metrics	Epic: Financial Metrics Stories: Display Financial Metrics, Data Collection, Visualization
3		Request (AI, Report, Hardware, API Connection,), Data Library	Epic: Stories: UI, File Upload Functionality, Data Parsing and Validation
4	Dashboard Visualization	Overview, Financial Metrics, Operational Metrics, etc.	Epic: Dashboard Visualization Stories: , Visualization Components, Data Retrieval
5	Export (pdf, csv)	Request (Report), Data Library	Epic: Export Functionality Stories: Export Options UI, Data Formatting Logic
6	Financial Plan Inputs	Finance (Planning, Goals List)	Epic: Financial Plan Inputs Stories: Input Forms, Data Processing, Validation
Continued on Next Page			

Pages Linked to Features Linked to Jira (2/2)

Feature Number	Feature	Pages	and Stories
Continued from Previous Page			
1	Operational Metrics	Overview, Operational Metrics	Epic: Operational Metrics Stories: Identify Metrics, Data Integration, Visualization
2	Data connection	Overview, Data Library	Epic: Data Connection Stories: External Data Sources, Data Connectors, Synchronization
3	AI-supported Data Verification	Request (AI), Data Library	Epic: AI-supported Data Verification Stories: Verification Criteria, AI Model Development, Integration
4	AI-supported Financial Planning	Finance (Planning)	Epic: AI-supported Financial Planning Stories: Planning Algorithms, AI Model Development, Scenario Analysis
5	Settings and Profile Permissions	Settings and Profile Permissions	Epic: Settings and Profile Permissions Stories: , , API Endpoints

Implementing in Jira

1. **Create Epics:**
   - Create an epic for each major feature.
   - Example: Epic "Cloud-Based Web Platform"
2. **Create Stories:**
   - Create stories under each epic to cover all functionalities detailed in the FSD.
   - Add the information to the fields in the story issue (e.g., time to complete, linked repository)
   - Example: Story "User Authentication and Authorization" under Epic "Cloud-Based Web Platform"
3. **Create Tasks/Sub-Tasks:**
   - Break down stories into smaller tasks or sub-tasks.
   - Add information to the fields in the tasks/sub-tasks issue (e.g., time to complete, linked repository)
   - Example: Tasks "Implement login/signup functionality" and "Setup user roles and permissions" under Story "User Authentication and Authorization"
4. **Link Stories to**
   - Use labels, com to indicate which pages each story impacts.
   - Example: Link Story "Display Financial Metrics" with Page Design "Overview Page"

Data Model

Data Model Summary (NoSQL Firestore)

- Ambient Systems uses FireBase NoSQL database called FireStore
- is a NoSQL, document-oriented database
- Unlike a SQL database, there are no tables or rows.
- Instead, you store data in *documents*, which are organized into *collections*
- Firebase consists of Collection, Document, and Fields
- Fields can support 9 different data types

More reading: https://firebase.google.com/docs/firestore/data-model

Data Types

Data type	Sort order	Notes
Array	By element values	An array cannot contain another array value as one of its elements. Within an array, elements maintain the position assigned to them. When sorting two or more arrays, arrays are ordered based on their element values. When comparing two arrays, the first elements of each array are compared. If the first elements are equal, then the second elements are compared and so on until a difference is found. If an array runs out of elements to compare but is equal up to that point, then the shorter array is ordered before the longer array. For example, [1, 2, 3] < [1, 2, 3, 1] < [2]. The array [2] has the greatest first element value. The array [1, 2, 3] has elements equal to the first three elements of [1, 2, 3, 1] but is shorter in length.
Boolean	false < true	—
Timestamp	Chronological	When stored in , precise only to microseconds; any additional precision is rounded down.
Number (Floating-point)	Numeric	64-bit double precision, IEEE 754.
Geographical point	By latitude, then longitude	At this time we do not recommend using this data type due to querying limitations. It is generally better to store latitude and longitude as separate numeric fields. If your app needs simple distance-based geoqueries, see Geo queries
Map	By keys, then by value	Represents an object embedded within a document. When indexed, you can query on subfields. If you exclude this value from indexing, then all subfields are also excluded from indexing. Key ordering is always sorted. For example, if you write {c: "foo", a: "bar", b: "qux"} the map is sorted by key and saved as {a: "bar", b: "qux", c: "foo"}. Map fields are sorted by key and compared by key-value pairs, first comparing the keys and then the values. If the first key-value pairs are equal, the next key-value pairs are compared, and so on. If two maps have all of the same key-value pairs, then map length is considered. For example, the following maps are in ascending order: {a: "aaa", b: "baz"} {a: "foo", b: "bar"} {a: "foo", b: "bar", c: "qux"} {a: "foo", b: "baz"} {b: "aaa", c: "baz"} {c: "aaa"}
Null	None	—
Reference	By path elements (collection, document ID, collection, document ID...)	For example, projects/[PROJECT_ID]/databases/[DATABASE_ID]/documents/[DOCUMENT_PATH].
Text string	UTF-8 encoded byte order	Up to 1,048,487 bytes (1 MiB - 89 bytes). Only the first 1,500 bytes of the UTF-8 representation are considered by queries.

Ambient Systems Data Model

- The data model involves a series of relationships that interconnect metrics, data sources, models, recommendations, and users.
- Ambient Systems data model plan consists of 8 ‘core’ collections and 6 ‘associative’ collections.
- Unknown relatedness is also possible, which occurs when we don’t know exactly which collection fields link with the other collection’s fields
- You can resolve this with two types of fields:
  - Optional Fields: For example, if you have an associative collection like ModelRecommendations but the related model or recommendation is not known, you can simply omit the modelId or recommendationId field.
  - Dynamic Fields: For more flexibility, you can use a map to dynamically store relationships. This approach allows you to add relationships as they become known.

Ambient Systems ‘Core’ Collections

Collection #	Collection	Description	Status	Creator
1	Alerts	User alerts	Started	User
2	Data Sources	Data sources to calculate metrics	Started	User
3	Metrics	Financial and operational metrics	Developed	Ambient
4	Users	Users and their preferences	Developed	User
5	Models (soon)	AI models	TBD	Ambient
6	Recommendations (soon)	Output from AI models	TBD	Ambient (Models Collection)
7	Storage Data Files (soon)	Files from data sources	TBD	User
8	Locations/Sites (soon)	User’s locations and sites where they want to calculate a metric.	TBD	User

Legend:
- TBD
- Designed
- Started
- Developed

Ambient Systems ‘Associative’ Collections

Collection #	Collection	Collection 1	Collection 2	“Join”	Owner
1	MetricDataSources	Metric	Data Sources	dataSourceId: String (reference to the data source) metricId: String (reference to the metric)	User
2	DataSourceLocations	Data Source	Site/Location	id: String (unique identifier for the link) dataSourceId: String (reference to the data source) locationId: String (reference to the location/site)	
3	ModelRecommendations	Model	Recommendation	id: String (unique identifier for the link) modelId: String (reference to the model) recommendationId: String (reference to the recommendation)	
4	UserRecommendations	User	Recommendation	id: String (unique identifier for the link) userId: String (reference to the user) recommendationId: String (reference to the recommendation)	
5	ModelDataSources	Data Source	Model	id: String (unique identifier for the link) modelId: String (reference to the model) dataSourceId: String (reference to the data source)	
6	RecommendationMetrics	Recommendation	Metric	id: String (unique identifier for the link) recommendationId: String (reference to the recommendation) metricId: String (reference to the metric)	
Probably more…					

Chrome File Edit View History Bookmarks Profiles Tab Window Help
Fri Jul 5 4:02 PM
console.firebase.google.com/project/ambient-fintech-platform/firestore/databases/-default-/data/~2Fmetrics~2Fg8SbPzoCxFOvDKhZ5zkn?view=panel-view&query=1%7CLIM%7C3%2F100&scopeType=collectio...

Firebase

Ambient FinTech

Project Overview

Generative AI

Build with Gemini NEW

Project shortcuts

App Hosting NEW

Firestore Database

Storage

Data Connect NEW

Authentication

Product categories

Build

Run

Analytics

All products

Spark
No-cost $0/month

Upgrade

Cloud Firestore

Add database

Data | Rules | Indexes | Usage | Extensions

Protect your Cloud Firestore resources from abuse, such as billing fraud or phishing Configure App Check

Panel view | Query builder

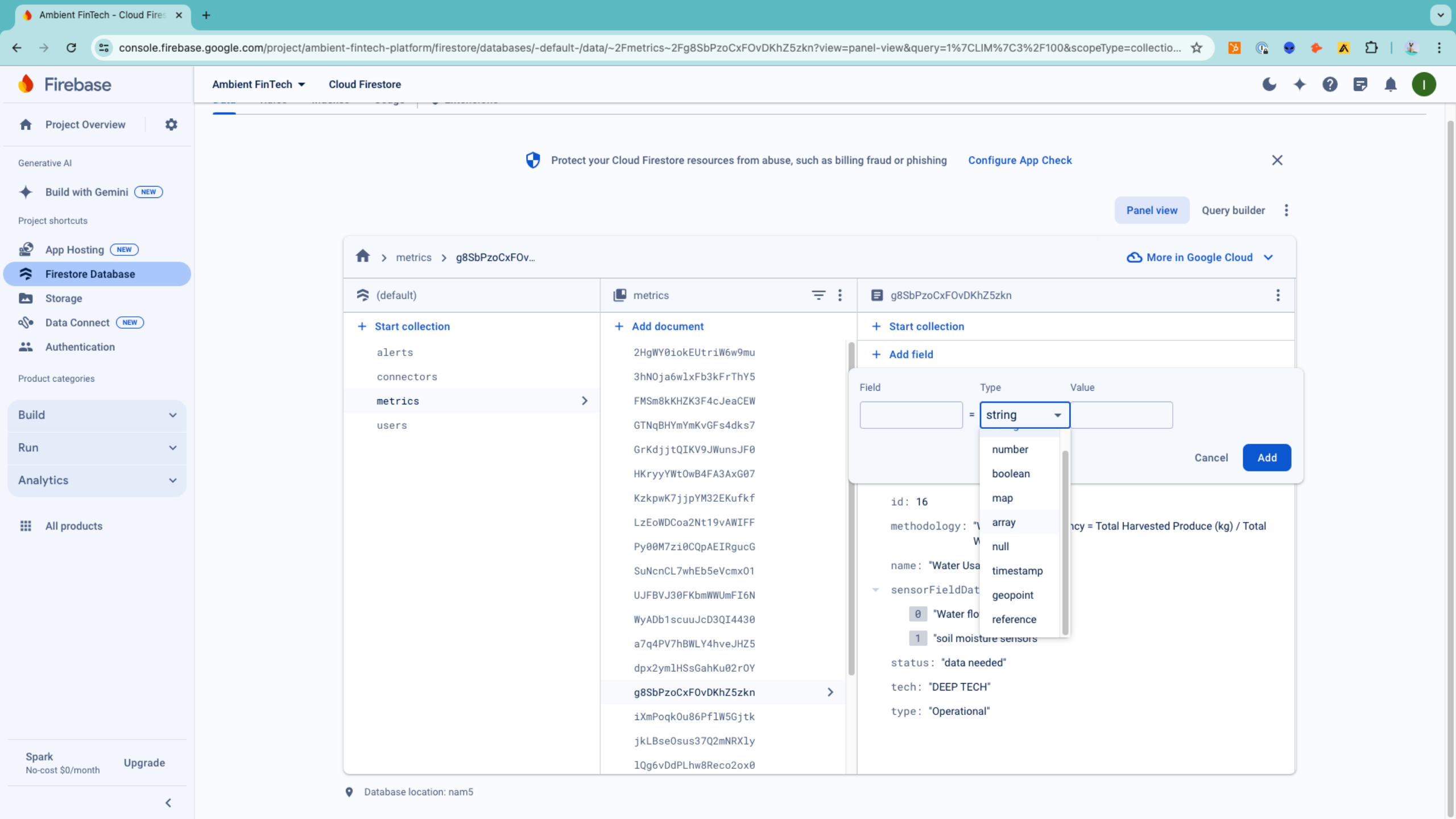
metrics > g8SbPzoCxFOv...

More in Google Cloud

(default)	metrics	g8SbPzoCxFOvDKhZ5zkn
+ Start collection	+ Add document	+ Start collection
alerts	2HgWY0iokEUtriW6w9mu	+ Add field
connectors	3hNOja6wlxFb3kFrThY5	category: "Vertical Farming"
metrics	FMSm8kKHZK3F4cJeaCEW	datasource
users	GTNqBHYmYmKvGFs4dks7	0 "Irrigation logs"
	GrKdjjtQIKV9JWunsJF0	1 "farm management systems"
	HKryyYWtOwB4FA3AxG07	description: "Amount of produce harvested per liter of water used."
	KzkpwK7jjpYM32EKufkf	id: 16
	LzEoWDCoa2Nt19vAWIFF	methodology: "Water Usage Efficiency = Total Harvested Produce (kg) / Total Water Used (liters)"
	Py00M7zi0CQpAEIRgucG	name: "Water Usage Efficiency"
	SuNcnCL7whEb5eVcmxO1	sensorFieldData
	UJFBVJ30FKbmWWUmFI6N	0 "Water flow meters"
	WyADb1scuuJcD3QI4430	1 "soil moisture sensors"
	a7q4PV7hBWLY4hveJHZ5	status: "data needed"
	dpx2ymlHSsGahKu02rOY	tech: "DEEP TECH"
	g8SbPzoCxFOvDKhZ5zkn	type: "Operational"
	iXmPoqkOu86PflW5Gjtk	
	jkLBseOsus37Q2mNRXly	

‘Metrics’ Collection Data Model

Field	Description	Field Data	Example
Metric_ID	Unique identifier of the metric.	Number	6
Name	Metric Name	Text string	Gross Rent Multiplier (GRM)
Description	Metric Description	Text string	“Property price divided by gross rental income, used for quick valuation."
Category	Industry for the metric.	Text string	Real Estate
~~Data source(s)~~			
Methodology	Methodology for calculating the metric.	Text string	GRM = Property Price / Gross Rental Income
SensorFieldData	Whether the metric uses sensor data from the field to be calculated (or video or footage).	Boolean	true
Status	Whether the metric is active or not.	Text string	‘data needed’
Tech	Subscription offering product type.	Text string	FinTech
Type	Metric type category.	Text string	Financial


Ambient FinTech - Cloud Fires
console.firebase.google.com/project/ambient-fintech-platform/firestore/databases/-default-/data/~2Fmetrics~2Fg8SbPzoCxFOvDKhZ5zkn?view=panel-view&query=1%7CLIM%7C3%2F100&scopeType=collectio...
Firebase
Ambient FinTech
Cloud Firestore
Project Overview
Generative AI
Build with Gemini NEW
Project shortcuts
App Hosting NEW
Firestore Database
Storage
Data Connect NEW
Authentication
Product categories
Build
Run
Analytics
All products
Spark
No-cost $0/month
Upgrade
Protect your Cloud Firestore resources from abuse, such as billing fraud or phishing
Configure App Check
Panel view
Query builder
metrics
g8SbPzoCxFOv...
More in Google Cloud
(default)
Start collection
alerts
connectors
metrics
users
metrics
Add document
2HgWY0iokEUtriW6w9mu
3hNOja6wlxFb3kFrThY5
FMSm8kKHZK3F4cJeaCEW
GTNqBHYmYmKvGFs4dks7
GrKdjjtQIKV9JWunsJF0
HKryyYWtOwB4FA3AxG07
KzkpwK7jjpYM32EKufkf
LzEoWDCoa2Nt19vAWIFF
Py00M7zi0CQpAEIRgucG
SuNcnCL7whEb5eVcmxO1
UJFBVJ30FKbmWWUmFI6N
WyADb1scuuJcD3QI4430
a7q4PV7hBWLY4hveJHZ5
dpx2ymlHSsGahKu02rOY
g8SbPzoCxFOvDKhZ5zkn
iXmPoqkOu86PflW5Gjtk
jkLBseOsus37Q2mNRXly
lQg6vDdPLhw8Reco2ox0
g8SbPzoCxFOvDKhZ5zkn
Start collection
Add field
Field
Type
Value
string
number
boolean
map
array
null
timestamp
geopoint
reference
Cancel
Add
id: 16
methodology:
name: "Water Usa
sensorFieldDat
0 "Water flo
1 "soil moisture sensors
status: "data needed"
tech: "DEEP TECH"
type: "Operational"
Database location: nam5

‘Data Sources’ Collection Data Model

Field	Description	Type	Example
Data Source ID	Unique identifier for the data source.		
Name	Name of the data source.		
Description	Description of the data source.		
industry	The industry this data source applies to.		
data_type	Type of data provided		e.g., "transaction", "sensor", "survey"
frequency	Frequency of data updates		e.g., "daily", "weekly", "monthly"
Data Edit date	TBD		
Data Type	TBD		
Link	Link to the storage location.		
File			
~~Method~~			
~~Metrics~~			
~~Required Column Mapping~~			
~~SiteID~~			
Status			
User Email			@ambientsystems.ai

‘Metric Data Source’ Collection

This collection will represent the many-to-many relationships between metrics and data sources.

- metric_data_source_id: Optional unique identifier.
- metric_id: Reference to the Metrics collection.
- data_source_id: Reference to the DataSources collection.
- calculation_method: Description of how the data source is used to calculate the metric (e.g., "summation", "average").
- weight: Optional field indicating the importance or contribution of the data source to the metric.

‘Recommendation’ Collection

Recommendations (soon)

- id: String (unique identifier for the recommendation)
- modelId: String (reference to the model that generated the recommendation)
- userId: String (reference to the user receiving the recommendation)
- title: String (title of the recommendation, e.g., "Equipment Optimization")
- description: String (description of the recommendation, e.g., "Cost savings from energy efficiency improvements")
- timeline: String (estimated timeline for the recommendation, e.g., "Short-term (1-3 months)")
- monthlyFinancialImpact: Number (estimated monthly financial impact, e.g., 1000)
- dataFeedsConnected: Number (number of connected data feeds, e.g., 2)
- totalDataFeeds: Number (total number of data feeds, e.g., 10)
- confidenceLevel: Number (confidence level of the recommendation as a percentage, e.g., 70)
- createdAt: Timestamp (time when the recommendation was created)

‘Models’ Collection

Recommendations (soon)

- id: String (unique identifier for the recommendation)
- modelId: String (reference to the model that generated the recommendation)
- userId: String (reference to the user receiving the recommendation)
- title: String (title of the recommendation, e.g., "Equipment Optimization")
- description: String (description of the recommendation, e.g., "Cost savings from energy efficiency improvements")
- timeline: String (estimated timeline for the recommendation, e.g., "Short-term (1-3 months)")
- monthlyFinancialImpact: Number (estimated monthly financial impact, e.g., 1000)
- dataFeedsConnected: Number (number of connected data feeds, e.g., 2)
- totalDataFeeds: Number (total number of data feeds, e.g., 10)
- progressLevel: Number (confidence level of the recommendation as a percentage, e.g., 70)
- createdAt: Timestamp (time when the recommendation was created)

Industry-Specific Fields: Transportation

Transportation:

- route_id: Identifier for a specific route.
- vehicle_type: Type of vehicle (e.g., "truck", "bus").
- fuel_type: Type of fuel used (e.g., "diesel", "electric").
- distance_covered: Total distance covered.
- emissions: Amount of emissions produced.

Industry-Specific Fields: Commercial Real Estate

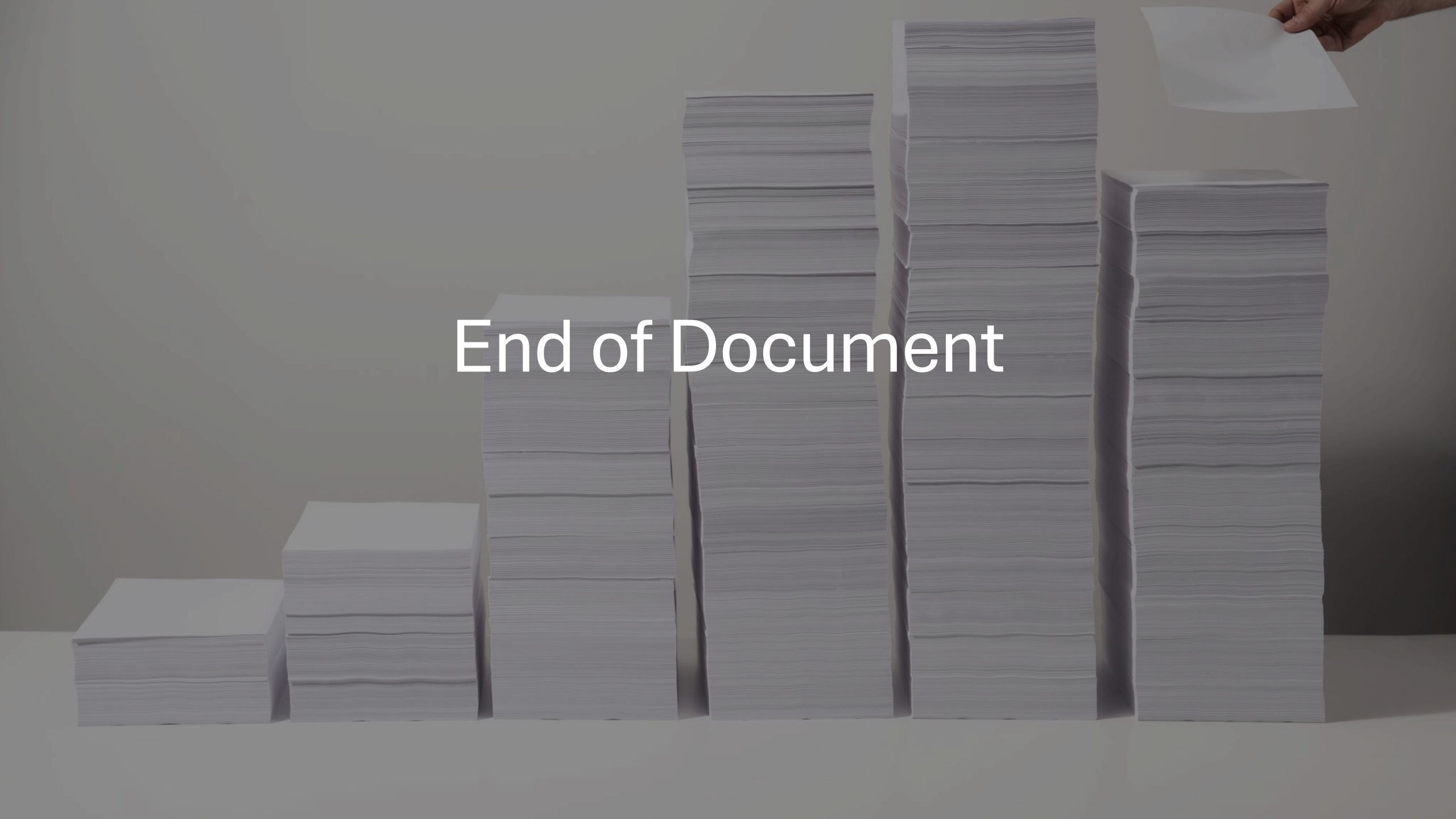
Commercial Real Estate:

- property_id: Identifier for the property.
- location: Location of the property.
- square_footage: Total square footage.
- occupancy_rate: Occupancy rate percentage.
- rent_per_sq_ft: Rent price per square foot.
- energy_consumption: Energy consumption data.

Industry-Specific Fields: Vertical Farming

Vertical Farming:

- farm_id: Identifier for the farm.
- crop_type: Type of crop grown.
- growth_cycle: Duration of the growth cycle.
- yield: Total yield produced.
- water_usage: Amount of water used.
- energy_usage: Amount of energy used.


End of Document